LEC - 60
SUMMARY
NOTES

```
import numpy as np
```

```
A bank account has the following transactions:

transactions = np.array([
     5000, -2000, 15000, -4500, 7000,
    -12000, 25000, -3000, 8000, -18000,
     4500, 22000, -6000, 9000, -2500
])
Positive values represent deposits and negative values represent withdrawals.
```

```
Extract deposits.
Extract withdrawals.
Calculate total deposits and withdrawals.
Calculate net transaction amount.
Calculate absolute transaction values.
```

```
transactions = np.array([
     5000, -2000, 15000, -4500, 7000,
    -12000, 25000, -3000, 8000, -18000,
     4500, 22000, -6000, 9000, -2500
])
```

```
# Extract deposits.

deposits = transactions[transactions > 0]
deposits
```

```
array([ 5000, 15000,  7000, 25000,  8000,  4500, 22000,  9000])
```

```
# Extract withdrawals.
withdrawals = transactions[transactions < 0]
withdrawals
```

```
array([ -2000,  -4500, -12000,  -3000, -18000,  -6000,  -2500])
```

```
# Calculate total deposits and withdrawals.

total_d = np.sum(deposits)
total_w = np.sum(withdrawals)
```

```
total_d
```

```
np.int64(95500)
```

```
total_w
```

```
np.int64(-48000)
```

```
# Calculate net transaction amount.
net_amt = np.sum(transactions)
net_amt
```

```
np.int64(47500)
```

```
# Calculate absolute transaction values.

abs_t = np.abs(transactions)
abs_t
```

```
array([ 5000,  2000, 15000,  4500,  7000, 12000, 25000,  3000,  8000,
       18000,  4500, 22000,  6000,  9000,  2500])
```

```
Find transactions whose absolute value is above the average absolute transaction.

Find the largest deposit.

Find the largest withdrawal.

Create labels "Credit" and "Debit"
```

```
Create labels "Credit" and "Debit".

Find percentage of transactions whose absolute value is greater than ₹10,000.

Reshape the data into 3 × 5.

Calculate total transaction value for each row.
```

```
# Find transactions whose absolute value is above the average absolute transaction.

avg_abs_t = np.mean(abs_t)
avg_abs_t
```

```
np.float64(9566.666666666666)
```

```
# abs_t[abs_t > avg_abs_t]

transactions[abs_t > avg_abs_t]
```

```
array([ 15000, -12000,  25000, -18000,  22000])
```

```
# Find the largest deposit.
np.max(transactions)
```

```
np.int64(25000)
```

```
# Find the largest withdrawal.
np.min(transactions)
```

```
np.int64(-18000)
```

```
# Create labels "Credit" and "Debit".

labels = np.where(transactions > 0, "Credit", "Debit")
labels
```

```
array(['Credit', 'Debit', 'Credit', 'Debit', 'Credit', 'Debit', 'Credit',
       'Debit', 'Credit', 'Debit', 'Credit', 'Credit', 'Debit', 'Credit',
       'Debit'], dtype='<U6')
```

```
# Find percentage of transactions whose absolute value is greater than ₹10,000.
abs10k = abs_t[abs_t > 10000]
abs10k
```

```
array([15000, 12000, 25000, 18000, 22000])
```

```
m = len(abs10k)
n = len(abs_t)

perc = m*100/n
perc
```

```
33.333333333333336
```

```
# Reshape the data into 3 × 5.
t_new = transactions.reshape(3,5)
t_new
```

```
array([[  5000,  -2000,  15000,  -4500,   7000],
       [-12000,  25000,  -3000,   8000, -18000],
       [  4500,  22000,  -6000,   9000,  -2500]])
```

```
# Calculate total transaction value for each row.

row_sm = np.sum(t_new, axis = 1)
row_sm
```

```
array([20500,     0, 27000])
```

```
A factory has four machines operating for six days:

production = np.array([
    [120, 135, 128, 145, 150, 138],
```

```
    [110, 125, 118, 130, 142, 135],
    [160, 155, 170, 165, 180, 175],
    [95, 105, 100, 115, 108, 120]
])
```

```
Calculate total production by machine.
Calculate total production by day.
Calculate average production of each machine.
Find the machine with the highest average production.
Find the day with the lowest production.
```

```
production = np.array([
    [120, 135, 128, 145, 150, 138],
    [110, 125, 118, 130, 142, 135],
    [160, 155, 170, 165, 180, 175],
    [95, 105, 100, 115, 108, 120]
])
```

```
machine_total_p = np.sum(production, axis = 1)
machine_total_p
```

```
array([ 816,  760, 1005,  643])
```

```
# Calculate total production by day.

day_total_p = np.sum(production, axis = 0)
day_total_p
```

```
array([485, 520, 516, 555, 580, 568])
```

```
# Calculate average production of each machine.
machine_avg = np.mean(production, axis = 1)
machine_avg
```

```
array([136.        , 126.66666667, 167.5       , 107.16666667])
```

```
# Find the machine with the highest average production.
ind = np.argmax(machine_avg)
ind
```

```
np.int64(2)
```

```
production[ind]
```

```
array([160, 155, 170, 165, 180, 175])
```

```
# Find the day with the lowest production.
day_total_p
```

```
array([485, 520, 516, 555, 580, 568])
```

```
np.argmin(day_total_p)
```

```
np.int64(0)
```

```
Find values above the overall average.
Increase production values below 120 by 5%.
Extract alternate days.
Extract the last 3 days for machines 1 and 3.
Create "High" and "Normal" labels based on the overall average.
```

```
# Find values above the overall average.
overall_avg = np.mean(production)
overall_avg
```

```
np.float64(134.33333333333334)
```

```
production[production > overall_avg]
```

```
array([135, 145, 150, 138, 142, 135, 160, 155, 170, 165, 180, 175])
```

```
# Increase production values below 120 by 5%.
production = np.where(production < 120, production*1.05, production)
production
```

```
array([[120.  , 135.  , 128.  , 145.  , 150.  , 138.  ],
       [115.5 , 125.  , 123.9 , 130.  , 142.  , 135.  ],
       [160.  , 155.  , 170.  , 165.  , 180.  , 175.  ],
       [ 99.75, 110.25, 105.  , 120.75, 113.4 , 120.  ]])
```

```
# Extract alternate days.
production[:,::2]
```

```
array([[120.  , 128.  , 150.  ],
       [115.5 , 123.9 , 142.  ],
       [160.  , 170.  , 180.  ],
       [ 99.75, 105.  , 113.4 ]])
```

```
production
```

```
array([[120.  , 135.  , 128.  , 145.  , 150.  , 138.  ],
       [115.5 , 125.  , 123.9 , 130.  , 142.  , 135.  ],
       [160.  , 155.  , 170.  , 165.  , 180.  , 175.  ],
       [ 99.75, 110.25, 105.  , 120.75, 113.4 , 120.  ]])
```

```
# Extract the last 3 days for machines 1 and 3.
production[[0,2]]
```

```
array([[120., 135., 128., 145., 150., 138.],
       [160., 155., 170., 165., 180., 175.]])
```

```
production[[0,2], -3:]
```

```
array([[145., 150., 138.],
       [165., 180., 175.]])
```

```
# Create "High" and "Normal" labels based on the overall average.

status = np.where(
    production > overall_avg,
    "High",
    "Normal"
)

print(status)
```

```
[['Normal' 'High' 'Normal' 'High' 'High' 'High']
 ['Normal' 'Normal' 'Normal' 'Normal' 'High' 'High']
 ['High' 'High' 'High' 'High' 'High' 'High']
 ['Normal' 'Normal' 'Normal' 'Normal' 'Normal' 'Normal']]
```

```
Consider the following loan portfolio:

loan_amount = np.array([
    100000, 250000, 500000, 750000,
    120000, 350000, 900000, 450000
])

interest_rate = np.array([
    8.5, 9.2, 8.8, 10.1,
    7.9, 9.5, 10.5, 8.7
])
```

```
Calculate annual interest for every loan.
Calculate total interest.
Calculate average interest.
Find the loan generating the highest interest.
Find loans above ₹400,000 with interest rate above 9%.
```

```
loan_amount = np.array([
    100000, 250000, 500000, 750000,
    120000, 350000, 900000, 450000
])
```

```
interest_rate = np.array([
    8.5, 9.2, 8.8, 10.1,
    7.9, 9.5, 10.5, 8.7
])
```

```
# Calculate annual interest for every loan.
annual_interest = (loan_amount*interest_rate)/100
annual_interest
```

```
array([ 8500., 23000., 44000., 75750.,  9480., 33250., 94500., 39150.])
```

```
# Calculate total interest.
total_int = np.sum(annual_interest)
total_int
```

```
np.float64(327630.0)
```

```
# Calculate average interest.
avg_int = np.mean(annual_interest)
avg_int
```

```
np.float64(40953.75)
```

```
# Find the loan generating the highest interest.
np.argmax(annual_interest)
```

```
np.int64(6)
```

```
# Find loans above ₹400,000 with interest rate above 9%.

criteria = (loan_amount > 400000) & (interest_rate > 9)
print(criteria)
loan_amount[criteria]
```

```
[False False False  True False False  True False]
array([750000, 900000])
```

HW

```
Categorize loans:
Rate >= 10 → High
Rate >= 9 → Medium
Otherwise → Low

Round interest to 2 decimal places.

Find median loan amount.

Find loans above median amount AND interest above average interest.

Calculate standard deviation of interest rates.
```

```
The coordinates of eight delivery locations are:

points = np.array([
    [3, 4],
    [5, 12],
    [8, 15],
    [7, 24],
    [9, 40],
    [12, 35],
    [15, 20],
    [6, 8]
])

Each row contains:

[x, y]

Calculate distance from the origin using:

distance = √(x² + y²)
```

```
Extract x and y coordinates.

Calculate distance for every location.

Round distances to 2 decimals.

Find nearest and farthest location.

Find average distance.

Find locations above average distance.

Categorize:
if >= 30 → Long
elseif >= 15 → Medium
Otherwise → Short


Find percentage of Long-distance deliveries.

Flatten the coordinate array.

Find the size of the flattened array.
```